

Министерство науки и высшего образования Российской
Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №2

По дисциплине Алгоритмы и структуры данных

Тема работы Сложность алгоритмов

Обучающийся Черненко Андрей Дмитриевич

Факультет Факультет прикладной информатики

Группа K4111c

Образовательная программа Разработка и эксплуатация
промышленного бэкенда

Обучающийся	_____	_____	<u>Черненко А. Д.</u>
	(дата)	(подпись)	(Ф.И.О.)

Руководитель	_____	_____	<u>Мусаев А. А.</u>
	(дата)	(подпись)	(Ф.И.О.)

Содержание

ВВЕДЕНИЕ	2
1 Графики сложности алгоритмов	2
1.1 Условие	2
1.2 Решение	2
1.3 Краткое сравнение сложностей	3
2 Задание 2: Программы с различной сложностью	5
2.1 Условие	5
2.2 Сводная таблица функций	5
2.3 Реализация и пояснения	5
2.3.1 Пара 1: $O(2n)$	5
2.3.2 Пара 2: $O(n \log n)$	7
2.3.3 Пара 3: $O(2n^3)$	7
2.3.4 Пара 4: $O(n!)$	9
2.3.5 Пара 5: $O(2 \log n)$	10

ВВЕДЕНИЕ

Целью данной лабораторной работы является изучение асимптотической сложности алгоритмов и практическое сравнение скорости роста их времени выполнения при увеличении размера входных данных. Понимание того, как ведёт себя алгоритм на больших объёмах информации, является ключевым навыком при разработке эффективных программных решений.

В ходе работы рассматриваются основные классы сложности — $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, $O(2^n)$, а также более специфические случаи: $O(2n)$, $O(n \log n)$, $O(2n^3)$, $O(n!)$ и $O(2 \log n)$.

В работе решаются следующие задачи:

1. построение графиков зависимости количества шагов алгоритма от размера входных данных для основных асимптотик и их сравнительный анализ;
2. разработка 10 функций, сгруппированных в 5 пар, каждая из которых решает различные задачи при одинаковой асимптотической сложности;
3. обоснование заявленной сложности для каждой функции на основе анализа структуры кода.

Все программы реализованы на языке C# с использованием .NET 8.

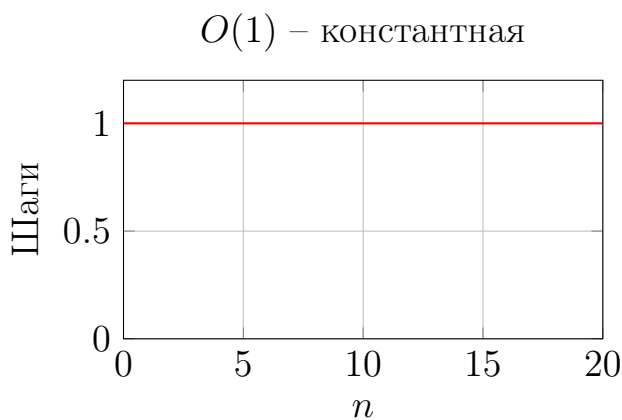
1 Графики сложности алгоритмов

1.1 Условие

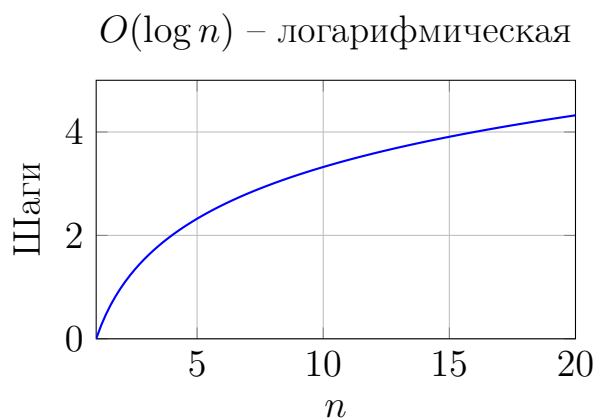
Нужно сравнить основные асимптотики, которые встречаются при анализе алгоритмов, и показать их поведение при увеличении размера входных данных.

1.2 Решение

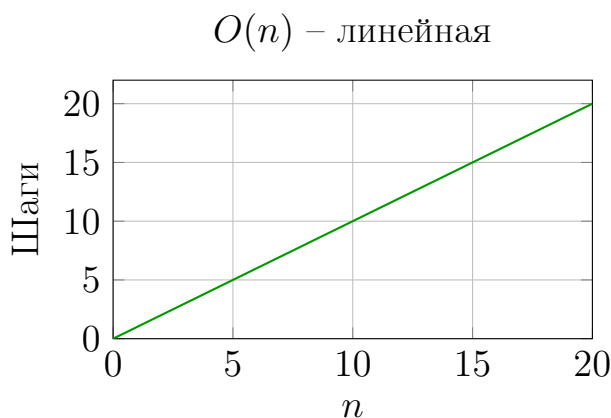
Для наглядности каждая функция сложности вынесена в отдельный график, чтобы все кривые были хорошо видны на своей шкале.



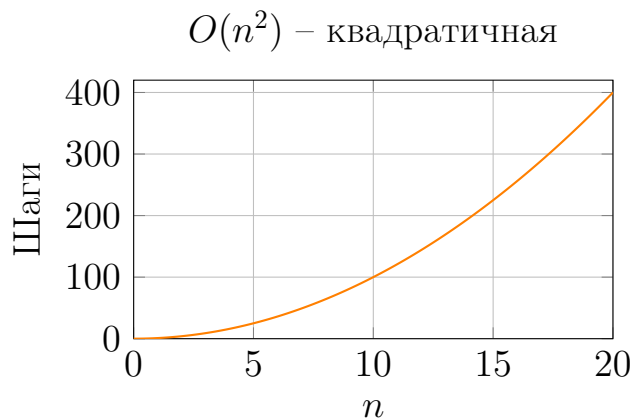
(a) Константная сложность



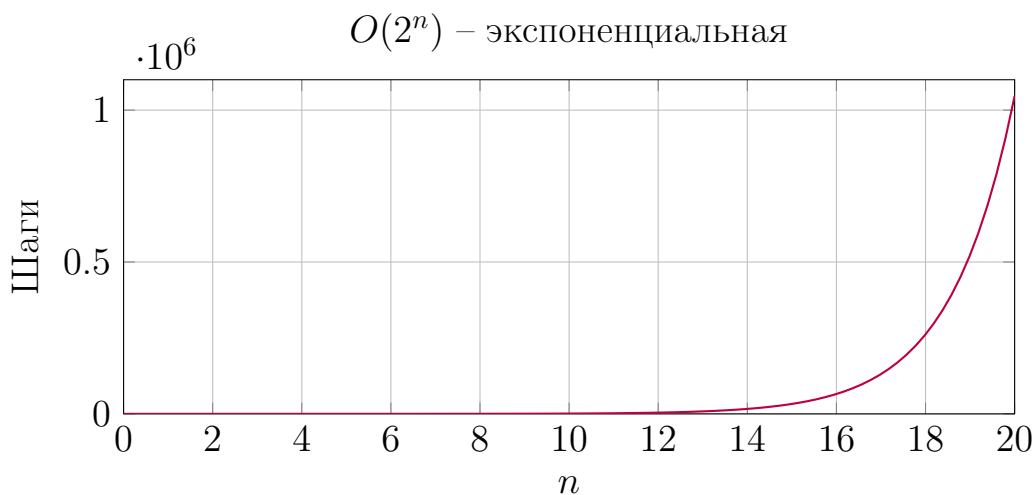
(b) Логарифмическая сложность



(c) Линейная сложность



(d) Квадратичная сложность



(e) Экспоненциальная сложность

Рисунок 1 – Отдельные графики основных асимптотик

1.3 Краткое сравнение сложностей

- $O(1)$ не зависит от размера входных данных и остаётся постоянной.
- $O(\log n)$ растёт очень медленно, поэтому подходит для эффективного

поиска.

- $O(n)$ увеличивается пропорционально размеру входа и считается линейной.
- $O(n^2)$ быстро становится дорогой при росте n , особенно на больших данных.
- $O(2^n)$ растёт наиболее резко и становится практически непригодной уже при умеренных значениях n .

2 Задание 2: Программы с различной сложностью

2.1 Условие

Необходимо создать 10 отдельных функций или блоков кода, сгруппированных в 5 пар. В каждой паре две программы должны иметь одинаковую асимптотическую сложность $O(n)$, но решать разные задачи или решать одну задачу разными способами. После написания кода требуется объяснить, почему в каждой программе именно та сложность, которая требовалась.

2.2 Сводная таблица функций

Таблица 1: Таблица функций и их сложностей

№ функции	Сложность $O(n)$	Аналогия из жизни
1, 2	$O(2n)$	Прочитать список покупок дважды: сначала проверить, что всё есть, потом посчитать общую сумму
3, 4	$O(n \log n)$	Отсортировать стопку писем по дате, чтобы найти самое старое
5, 6	$O(2n^3)$	Перебрать все тройки ингредиентов для двух разных блюд из одного набора продуктов
7, 8	$O(n!)$	Составить все возможные порядки выступления музыкантов на концерте
9, 10	$O(2 \log n)$	Угадать два загаданных числа, каждый раз спрашивая «больше или меньше?»

2.3 Реализация и пояснения

2.3.1 Пара 1: $O(2n)$

```
1 public void ProcessShoppingList(List<Product> shoppingList)
```

```

2 {
3     //                                     :
4     foreach (var p in shoppingList)
5     {
6         if (!p.InStock) Console.WriteLine($"    {p.Name}
7                                     ");
8     }
9     //                                     :
10    decimal total = 0;
11    foreach (var p in shoppingList)
12    {
13        if (p.InStock) total += p.Price;
14    }
15    Console.WriteLine($"    : {total}");
16 }

```

Листинг 1 – Функция 1. Обработка списка покупок за два прохода

Пояснение. В функции ровно два независимых цикла `foreach`, каждый из которых проходит по всем n элементам списка. Первый цикл выполняет константное число операций на элемент, второй — тоже. Итоговая сложность: $O(n) + O(n) = O(2n)$.

```

1 public void FindMinMaxPrices(List<Product> products)
2 {
3     //                                     :
4     decimal max = decimal.MinValue;
5     foreach (var p in products)
6     {
7         if (p.Price > max) max = p.Price;
8     }
9
10    //                                     :
11    decimal min = decimal.MaxValue;
12    foreach (var p in products)
13    {
14        if (p.Price < min) min = p.Price;
15    }
16
17    Console.WriteLine($"    : {min},    : {max}");
18 }

```

Листинг 2 – Функция 2. Поиск минимальной и максимальной цены

Пояснение. Аналогично предыдущей функции: два последовательных прохода по списку из n товаров. Каждый цикл выполняет $O(n)$ операций, вме-

сте — $O(2n)$. Задачи разные (проверка наличия и суммирование vs поиск минимума и максимума), но сложность одинаковая.

2.3.2 Пара 2: $O(n \log n)$

```
1 public void SortLettersByDate(List<Letter> letters)
2 {
3     letters.Sort((a, b) => a.Date.CompareTo(b.Date));
4     foreach (var letter in letters)
5     {
6         Console.WriteLine($" {letter.Sender} - {letter.Date:dd.MM.yyyy}")
7         ;
8     }
9 }
```

Листинг 3 – Функция 3. Сортировка писем по дате

Пояснение. Метод `List.Sort` в C# реализует алгоритм Introspective Sort (комбинация QuickSort, HeapSort и InsertionSort), средняя и худшая сложность которого — $O(n \log n)$, где n — количество писем. Вывод после сортировки добавляет лишь $O(n)$, что не меняет итоговую асимптотику.

```
1 public void SortLettersBySender(List<Letter> letters)
2 {
3     letters.Sort((a, b) => string.Compare(a.Sender, b.Sender,
4         StringComparison.Ordinal));
5     foreach (var letter in letters)
6     {
7         Console.WriteLine($" {letter.Sender} - {letter.Date:dd.MM.yyyy}")
8         ;
9     }
10 }
```

Листинг 4 – Функция 4. Сортировка писем по имени отправителя

Пояснение. Используется тот же алгоритм сортировки, что и в функции 3, поэтому сложность также $O(n \log n)$. Задача другая — сортировка по строковому полю, а не по дате, но асимптотика остаётся прежней.

2.3.3 Пара 3: $O(2n^3)$

```
1 public void GenerateDishCombinations(List<Ingredient> ingredients)
2 {
3     // : 1
4     Console.WriteLine(" 1:");
```

```

5     for (int i = 0; i < ingredients.Count; i++)
6         for (int j = i + 1; j < ingredients.Count; j++)
7             for (int k = j + 1; k < ingredients.Count; k++)
8                 Console.WriteLine($"    {ingredients[i].Name} + {
9                     ingredients[j].Name} + {ingredients[k].Name}");
10
11 //                                     :                                     2
12 Console.WriteLine("                                     2:");
13 for (int i = 0; i < ingredients.Count; i++)
14     for (int j = i + 1; j < ingredients.Count; j++)
15         for (int k = j + 1; k < ingredients.Count; k++)
16             Console.WriteLine($"    {ingredients[i].Name} + {
17                 ingredients[j].Name} + {ingredients[k].Name}");
18 }

```

Листинг 5 – Функция 5. Комбинации ингредиентов для двух блюд

Пояснение. В функции два последовательных блока, каждый из которых содержит три вложенных цикла. Каждый блок перебирает все тройки элементов, что даёт $O(n^3)$ операций. Два таких блока вместе дают $O(n^3) + O(n^3) = O(2n^3)$.

```

1 public void CheckTripletsForPromotions(List<Product> products)
2 {
3     //                                     :                                     1 (                                     > 1000)
4     Console.WriteLine("                                     1 (                                     > 1000):");
5     for (int i = 0; i < products.Count; i++)
6         for (int j = i + 1; j < products.Count; j++)
7             for (int k = j + 1; k < products.Count; k++)
8             {
9                 decimal sum = products[i].Price + products[j].Price +
10                     products[k].Price;
11                 if (sum > 1000) Console.WriteLine($"    {products[i].Name
12                     }, {products[j].Name}, {products[k].Name} = {sum}");
13             }
14
15 //                                     :                                     2 (                                     < 500)
16 Console.WriteLine("                                     2 (                                     < 500):");
17 for (int i = 0; i < products.Count; i++)
18     for (int j = i + 1; j < products.Count; j++)
19         for (int k = j + 1; k < products.Count; k++)
20         {
21             decimal sum = products[i].Price + products[j].Price +
22                 products[k].Price;
23             if (sum < 500) Console.WriteLine($"    {products[i].Name},
24                 {products[j].Name}, {products[k].Name} = {sum}");
25         }
26 }

```

Листинг 6 – Функция 6. Проверка троек товаров для двух акций

Пояснение. Структура полностью аналогична функции 5: два блока по три вложенных цикла. Каждый блок даёт $O(n^3)$, итого — $O(2n^3)$. Задача отличается только условием фильтрации троек (две разные акции).

2.3.4 Пара 4: $O(n!)$

```
1 public void GeneratePerformanceOrders(List<Musician> musicians, int left,
2   int right)
3 {
4     if (left == right)
5     {
6         Console.WriteLine("  " + string.Join(" -> ", musicians.Select(m =>
7           m.Name)));
8     }
9     else
10    {
11        for (int i = left; i <= right; i++)
12        {
13            Swap(musicians, left, i);
14            GeneratePerformanceOrders(musicians, left + 1, right);
15            Swap(musicians, left, i);
16        }
17    }
18 }
```

Листинг 7 – Функция 7. Все порядки выступления музыкантов

Пояснение. Рекурсивная генерация всех перестановок списка музыкантов. На первом уровне рекурсии доступно n вариантов, на втором — $n - 1$, на третьем — $n - 2$, и так далее. Общее количество листьев дерева рекурсии равно $n!$, поэтому сложность — $O(n!)$.

```
1 public void GenerateBusSeating(List<Musician> musicians, int left, int
2   right)
3 {
4     if (left == right)
5     {
6         Console.WriteLine("  " + string.Join(", ", musicians.Select(m => m
7           .Name)));
8     }
9     else
10    {
11        for (int i = left; i <= right; i++)
```

```

10     {
11         Swap(musicians, left, i);
12         GenerateBusSeating(musicians, left + 1, right);
13         Swap(musicians, left, i);
14     }
15 }
16 }

```

Листинг 8 – Функция 8. Все порядки рассадки музыкантов в автобусе

Пояснение. Алгоритм идентичен функции 7 — генерация всех перестановок, поэтому сложность также $O(n!)$. Отличие только в семантике задачи: рассадка в автобусе вместо выступления на сцене.

2.3.5 Пара 5: $O(2 \log n)$

```

1 public void GuessTwoNumbers(int[] sortedRange, int target1, int target2)
2 {
3     Console.WriteLine($"                {target1}:");
4     BinarySearch(sortedRange, target1);
5     Console.WriteLine($"                {target2}:");
6     BinarySearch(sortedRange, target2);
7 }
8
9 private void BinarySearch(int[] arr, int target)
10 {
11     int left = 0, right = arr.Length - 1;
12     while (left <= right)
13     {
14         int mid = left + (right - left) / 2;
15         if (arr[mid] == target) { Console.WriteLine($"                :
16                                     {target}"); return; }
17         if (arr[mid] < target) left = mid + 1;
18         else right = mid - 1;
19     }
20     Console.WriteLine($"                {target}                ");
21 }

```

Листинг 9 – Функция 9. Поиск двух загаданных чисел бинарным поиском

Пояснение. Функция вызывает классический бинарный поиск ровно два раза. Каждый вызов имеет сложность $O(\log n)$, где n — размер отсортированного массива. Итоговая сложность: $O(\log n) + O(\log n) = O(2 \log n)$.

```

1 public void FindInsertionPoints(int[] sortedRange, int value1, int value2)
2 {
3     Console.WriteLine($"                {value1}:");

```

```

4     BinarySearchInsertion(sortedRange, value1);
5     Console.WriteLine($"                {value2}:");
6     BinarySearchInsertion(sortedRange, value2);
7 }
8
9 private void BinarySearchInsertion(int[] arr, int value)
10 {
11     int left = 0, right = arr.Length;
12     while (left < right)
13     {
14         int mid = left + (right - left) / 2;
15         if (arr[mid] < value) left = mid + 1;
16         else right = mid;
17     }
18     Console.WriteLine($"                {value}:
19         {left}");
20 }

```

Листинг 10 – Функция 10. Поиск точек вставки для двух новых чисел

Пояснение. Аналогично функции 9: два вызова бинарного поиска (в данном случае — поиска нижней границы, lower bound). Каждый вызов выполняется за $O(\log n)$, итого — $O(2 \log n)$. Задача отличается тем, что ищется не сам элемент, а позиция, куда его нужно вставить.